

UNIVERSIDADE SÃO JUDAS TADEU

Bruna Izabel U. Tony – 312410474

Miguel Henrique Semensato – 312411876

Fernando Ribeiro Todaro – 312411785

Arthur Queiroz da Silva – 312310852

Gustavo De Souza Fernandes – 312321691

Rodrigo Mendola Bassini – 312412355

Danilo Cassiano de Souza – 312610948

Secure Code

São Bernardo do Campo – SP

2026

DOCUMENTAÇÃO DO PROJETO SECURECODEAUDIT

1. Introdução

O SecureCodeAudit é um protótipo de analisador estático (Linter) desenvolvido para a linguagem SecureCode. Seu objetivo é identificar automaticamente vulnerabilidades e padrões inseguros através da definição e execução de regras de auditoria. O sistema engloba as fases clássicas de um compilador (Análise Léxica, Sintática e Semântica) e culmina em um Motor de Inferência capaz de ler logs dinâmicos para auditoria.

2. Objetivo do Sistema

O objetivo principal do projeto é permitir a criação de regras de segurança que possam ser utilizadas para verificar automaticamente comportamentos considerados inseguros em um conjunto de dados.

Entre as funcionalidades do sistema estão:

- Definição de regras de auditoria
- Classificação de criticidade das regras
- Validação sintática da linguagem
- Validação semântica das regras criadas
- Armazenamento das regras em tabela de símbolos
- Execução automática das auditorias
- Emissão de alertas de segurança

3. Estrutura do Projeto

O sistema é composto pelos seguintes módulos:

Classe	Responsabilidade
Principal	Coordena a execução do sistema
AnalizadorLexico	Realiza a análise léxica
Parser TabelaSimbolos	Realiza a análise sintática Armazena as regras declaradas
Regra	Representa uma regra de auditoria
RegistroTeste	Carrega os dados de teste
MotorAuditoria	Executa as auditorias
Token	Representa os tokens gerados

4. Fluxo Geral de Funcionamento

O funcionamento do sistema segue a sequência apresentada abaixo:

Código SecureCode



Análise Léxica



Geração de Tokens



Análise Sintática



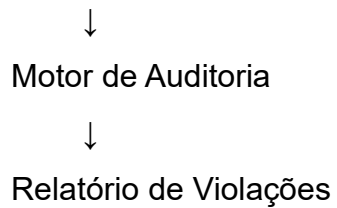
Validação Semântica



Tabela de Símbolos



Leitura dos Dados de Teste



5. Análise Léxica e Autômatos Finitos

A etapa de tokenização do código-fonte foi implementada utilizando o gerador **JFlex**, que converte Expressões Regulares em um Autômato Finito Determinístico (AFD). O autômato descarta comentários e espaços em branco, isolando os padrões válidos da linguagem. As principais expressões utilizadas foram:

- Identificadores (ID): `[A-Z_][A-Z0-9_]*`
- Strings Literais: `"[^"]*"`
- Comentários de Linha: `"//"[^\\n]*`

O analisador léxico reconhece:

Palavras reservadas

AUDITORIA
DEFINIR
REGRA
EXECUTAR
FIM

Níveis de criticidade

BAIXA
MÉDIA
ALTA
CRITICA

Operadores

EQUALS

NOT

CONTAINS

MATCHES

Ações

REJEITAR

NOTIFICAR

SOLICITAR_MFA

LOG_ERROR

6. Análise Sintática

A análise sintática foi implementada utilizando um **Parser Descendente Recursivo**. Este método consome a lista de tokens gerada pelo JFlex e valida a estrutura da linguagem estritamente com base na seguinte Gramática EBNF oficial:

`<projeto> ::= "AUDITORIA" <id> "DEFINIR" <regras> "EXECUTAR" "FIM"`

`<regras> ::= ( <regra_unitaria> )+`

`<regra_unitaria> ::= "REGRA" <id> "NIVEL" <prioridade> "SE"`
`<condicao_seguranca> "ACAO" <resultado> ";"`

`<prioridade> ::= "BAIXA" | "MEDIA" | "ALTA" | "CRITICA"`

`<condicao_seguranca> ::= <termo_seg> ( ( "AND" | "OR" ) <termo_seg> )*`

`<termo_seg> ::= <chave> <operador> <valor>`

`<chave> ::= "TOKEN" | "SCOPE" | "USER_ROLE" | "FILE_TYPE" | "PERMISSION"`

`<operador> ::= "CONTAINS" | "EQUALS" | "MATCHES" | "NOT"`

<valor> ::= "" [^"]* ""

<resultado> ::= "REJEITAR" | "NOTIFICAR" | "SOLICITAR_MFA" | "LOG_ERROR"

<id> ::= [A-Z_][A-Z0-9_]*

Exemplo de estrutura válida:

```
11
12 // Um scrip na linguagem EBNF
13 String codigoSecureCode = ""
14     AUDITORIA SECURE_CODE
15     DEFINIR
16         REGRA REGRA_ACESSO_ADMIN NIVEL CRITICA SE USER_ROLE EQUALS "admin" ACAO REJEITAR ;
17         REGRA REGRA_FOTO_PERFIL NIVEL BAIXA SE FILE_TYPE NOT "png" ACAO REJEITAR ;
18     EXECUTAR
19     FIM
20     "";
```

Caso algum elemento esteja fora da ordem esperada, o sistema gera um erro sintático e interrompe a execução.

7. Análise Semântica e Tabela de Símbolos

A validação utiliza uma estrutura de dados **HashSet** no Java para implementar a Tabela de Símbolos. Esta estrutura registra o identificador de cada regra no momento de sua declaração. Se houver tentativa de declarar duas regras com o mesmo nome, o sistema barra a compilação garantindo a integridade do escopo. Além disso, o analisador semântico valida inconsistências de negócio (ex: impedir que uma regra com prioridade BAIXA dispare uma ação agressiva de REJEITAR).

A tabela possui as seguintes funções:

- Registrar todas as regras criadas
- Evitar duplicidade de nomes
- Permitir consultas durante a auditoria

Exemplo:

```
14 public void adicionarRegra(String nomeRegra, int linha) {
15     if (regrasDeclaradas.contains(nomeRegra)) {
16         throw new RuntimeException("Erro Semântico [Linha " + linha + "]: A regra '" + nomeRegra + "' já foi declarada anteriormente neste escopo!");
17     }
18 }
```

Caso uma regra seja declarada mais de uma vez, o sistema gera um erro semântico.

8. Carregamento dos Dados de Teste

Após a validação das regras, o sistema carrega os dados presentes no arquivo de teste.

Exemplo:

USER_ROLE=admin

FILE_TYPE=jpg

Os dados são convertidos para uma estrutura do tipo chave-valor.

Exemplo:

```
{  
  USER_ROLE = admin  
  FILE_TYPE = jpg  
}
```

Essas informações serão utilizadas pelo Motor de Auditoria.

9. Motor de Inferência e Operadores

O Motor de Inferência é responsável por executar as regras cadastradas.

Cada regra é comparada com os dados carregados do arquivo de teste.

Exemplo de regra:

REGRA REGRA_ACESSO_ADMIN

NIVEL CRITICA

SE USER_ROLE EQUALS "admin"

ACAO REJEITAR

Exemplo de dado:

USER_ROLE=admin

Comparação realizada:

admin == admin

Como a condição foi satisfeita, a regra é acionada.

Resultado:

[ALERTA]

Violação identificada na regra:

REGRA_ACESSO_ADMIN

O sistema suporta os seguintes operadores:

EQUALS

Verifica igualdade exata (exemplo: USER_ROLE EQUALS "admin").

NOT

Verifica diferença (exemplo: FILE_TYPE NOT "png").

CONTAINS

Verifica se um texto contém um valor específico.

MATCHES

Realiza validações complexas utilizando expressões regulares.

10. Exemplo Completo de Execução

Código SecureCode De Entrada

AUDITORIA SECURE_CODE

DEFINIR

REGRA REGRA_ACESSO_ADMIN

NIVEL CRITICA

SE USER_ROLE EQUALS "admin"

ACAO REJEITAR ;

REGRA REGRA_FOTO_PERFIL

NIVEL BAIXA

SE FILE_TYPE NOT "png"

ACAO REJEITAR ;

EXECUTAR

FIM

Dados de Entrada Dinâmicos

USER_ROLE=admin

FILE_TYPE=jpg

Resultado Gerado No Terminal

```
--- Executando Regras contra o Registro ---  
[ALERTA] Violacao identificada na regra: REGRA_ACESSO_ADMIN  
-> Nivel: CRITICA | Acao requerida: REJEITAR  
[ALERTA] Violacao identificada na regra: REGRA_FOTO_PERFIL  
-> Nivel: BAIXA | Acao requerida: REJEITAR  
-----
```

11. Conclusão

O desenvolvimento da linguagem SecureCode Pro permitiu a aplicação prática da teoria de Linguagens Formais, Autômatos e Compiladores. A construção das três fases (léxica com JFlex, sintática com EBNF e semântica com HashSet) aliada à engenharia de software do motor de inferência resultou em uma ferramenta altamente didática e tecnicamente viável para a análise estática de conformidade e cibersegurança.